A Deep Reinforcement Learning Approach for Joint Task Offloading, Caching, and User Association in Mobile Edge Computing

Tasnia Ashrafi Heya
University of Dayton
Dayton, OH 45469, USA
Email: tashrafiheya1@udayton.edu

Sayed Erfan Arefin
University of Dayton
Dayton, OH 45469, USA
Email: sarefin1@udayton.edu

2025 10th International Conference on Fog and Mobile Edge Computing (FMEC) | 979-8-3315-4424-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/FMEC65595.2025.11119361

***Abstract*—Despite advances in the growing popularity of computation-intensive mobile applications, the limited computational capability of mobile devices has been highlighted over time. Mobile edge computing (MEC) addresses these challenges in optimizing task-offloading and user association while minimizing latency. Efficient task offloading is hindered by dynamic cache replacement utilizing edge node loads and limited small-cell base station (SBS) resources, leading to congestion and service delays. To address these challenges, we propose a Deep Q-Network (DQN)-based caching replacement approach that dynamically manages edge node loads and optimizes user association. By leveraging Markov Decision Processes (MDP) and Bernoulli and Zipf distributions, our method predicts content requests and optimally assigns tasks according to channel quality. This decentralized, reinforcement learning-driven model enables real-time task offloading without requiring complete system knowledge. Moreover, we integrate game-theoretic concepts from Guo et al.'s two-dimensional weakly acyclic game, which optimizes caching under SBS resource constraint, reducing cost and user-association for task offloading. We compared our approach with the state-of-the-art and show how our model enhances Quality of Service (QoS), reduces congestion, and improves scalability in ultra-dense MEC environments.**

***Index Terms*—Mobile Edge Computing, Deep Reinforcement Learning, Task Offloading, Cache Optimization, Deep Q-Network, User Association, Resource Allocation**

I. Introduction

The increasing demand for computation-intensive mobile applications, such as 3D gaming and face recognition, has exposed the computational limitations of mobile devices. Although mobile device hardware continues to advance, they often cannot efficiently handle resource-heavy tasks locally. To bridge this gap, Mobile Edge Computing (MEC) has therefore emerged as a promising solution, enabling task offloading to edge nodes to improve execution efficiency while maintaining a high Quality of Experience (QoE). However, designing an effective distributed offloading strategy remains challenging due to dynamic load variations across edge nodes, which are influenced by multiple mobile devices making offloading decisions at the same time. A key challenge in MEC is determining an optimal user association and task offloading strategy that prevents edge node overload while maintaining efficiency. This raises two fundamental questions:

- **Offloading Decision**: Should a mobile device offload a particular task or process it locally?
- **User-Edge Association & Resource Allocation**: If offloading is beneficial, which edge node should handle the task while balancing system load?

Traditional approaches, including deep reinforcement learning (DRL)-based methods, have primarily focused on offload decisions without comprehensively addressing edge node resource constraints. When multiple devices offload to the same node, resource contention leads to processing delays or task failures when deadlines expire. While some centralized algorithms are proposed to optimize these decisions, they often require full system knowledge, posing privacy and scalability concerns. To address these challenges, this work explores a caching replacement-driven task offloading approach that dynamically manages edge node loads while optimizing user association. Instead of merely deciding where to offload tasks, our approach leverages caching mechanisms to prioritize computation dynamically. By replacing cached tasks with higher-priority computations, the system ensures efficient resource utilization, minimizes delays, and prevents node overload. Given the high-dimensional nature of MEC environments, we employ a Deep Q-Network (DQN)-based approach to learn optimal caching and offloading policies. Leveraging a Markov Decision Process (MDP) with Bernoulli and Zipf distributions, each Access Point (AP) autonomously determines user associations based on channel quality, ensuring more efficient resource allocation. Additionally, Guo et al.'s [1] weakly acyclic game-based model, optimizes caching and user association under small-cell base station (SBS) constraints. By refining this approach with reinforcement learning-driven optimization, our approach further enhances QoS while managing edge resources in a scalable, privacy-preserving manner.

II. Related Work

This section reviews existing research on caching replacement-driven task offloading in mobile edge computing (MEC), with a focus on optimizing edge node load management and user association. The review is categorized into three main areas: Deep Reinforcement Learning (DRL) Approaches

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.

for Task Offloading, Caching Replacement Strategies, and Game-Theoretic Approaches for Load Balancing.

A. DRL-Based Approaches for Task Offloading

Several studies have leveraged deep reinforcement learning (DRL) techniques to optimize task offloading in MEC systems. Tang et al. [2] proposed a deep reinforcement learning-based task offloading approach, where a neural network predicts the optimal edge server for offloading based on the system state and available resources. Their method reduces completion time, improves resource utilization, and decreases energy consumption. However, a major limitation of this approach is its scalability issue—as the number of base stations (BSs) increases, the state space grows exponentially, making traditional reinforcement learning methods impractical. To overcome such challenges, [3] introduced a deep Q-network (DQN)-based approach, which enhances performance by reducing content caching delay in ultra-dense MEC systems. They consider a system where mobile devices have wireless charging capabilities and formulates the computation offloading problem as a Markov Decision Process (MDP). A deep neural network is used as a function approximator to manage the large state space, enabling an online strategic computation offloading policy. This solution is particularly useful in ultra-dense MEC environments where mobile users make decentralized offloading decisions without requiring complete system knowledge. Several other studies [3], [4], [5], [6], [7] have investigated distributed task offloading algorithms, allowing mobile devices to make offloading decisions independently based on local observations. However, designing an efficient distributed offloading algorithm remains challenging due to the dynamic nature of edge loads—as multiple devices offload tasks simultaneously, the resource availability at edge nodes fluctuates, leading to potential delays or task drops.

B. Caching Replacement Strategies in MEC

Caching plays a critical role in reducing network congestion and improving response time in MEC. [8] explored optimal request routing and content caching in heterogeneous networks to minimize content access delays. The study considered two models:

1) Congestion-sensitive model, where delay depends on user request load
2) Congestion-insensitive model, where delay remains constant regardless of user load

The problem was proven to be NP-hard except in special cases, such as when each content piece is requested by only one user or when there are at most two caches. The authors formulated the problem as a monotone submodular function maximization under matroid constraints and proposed a greedy algorithm for optimized caching and routing [8]. In a similar approach like our proposed model, [3] proposed a DQN-based caching policy to handle increased mobile data demand and prevent resource congestion at edge nodes. While mobile devices continue to evolve, edge resources remain limited, causing delays or task failures when nodes become overloaded [4], [5]. Reinforcement learning has been suggested as a viable solution to optimize caching replacement and minimize the loss of tasks in MEC systems. An alternative approach to task offloading and caching replacement is using game theory to model user-edge node association and optimize resource allocation. [1] focused on maximizing the number of mobile users (MUs) served by small base stations (SBSs) to reduce the macro base station (MBS) workload, leading to lower energy consumption and faster transmission rates. Since the joint optimization of caching and MU-BS association is NP-hard, the problem was decomposed into two subproblems: a) Content caching optimization, b)MU-BS association optimization.

The authors of [1] modeled this as a two-dimensional weakly acyclic game, which was solved using the Two-Dimensional-Action Iteration Algorithm (2DIA) to reach a Two-Dimensional Nash Equilibrium (2D-NE). The study also introduced a Bisection-based Optimal Power Control Algorithm to optimize transmission power, improving overall efficiency. However, a key limitation of this approach is its computational cost in dense SBS deployments. To mitigate this, the authors suggest offloading algorithm execution to the cloud, allowing algorithms to run virtually rather than directly on SBSs. This method ensures faster convergence times, as the powerful cloud infrastructure handles computations, minimizing the real-time burden on edge devices. While existing DRL-based, caching replacement, and game-theoretic approaches have significantly improved task offloading efficiency, challenges remain in dynamic user association and edge resource management. Most current models either:

- Do not fully adapt to fluctuating edge loads, leading to delays and task failures.
- Require global system knowledge, making them less scalable for real-world deployments.
- Struggle with high-dimensional state spaces, increasing computational overhead.

We build on these ideas by unifying caching replacement, dynamic edge load management, and user association into a single framework. Unlike previous efforts, our approach specifically integrates:

1) Deep Q-Network (DQN) for adaptive caching strategies
2) Decentralized decision-making to handle dynamic resource availability
3) Game-theoretic optimizations to improve scalability in ultra-dense MEC environments

This approach aims to reduce content access delay, improve load balancing at edge nodes, and ensure efficient task execution without requiring centralized control or complete system knowledge.

III. Background

A. State-of-the-art: Task-offloading

In their paper, Tang et al. [2] propose a deep reinforcement learning (DRL) approach that enables mobile devices to make autonomous decisions about offloading tasks to a base station based on their local observations. Their goal is to address

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.

the unpredictable load behavior at edge nodes in a Mobile Edge Computing (MEC) system by developing two distinct algorithms: one for decision-making at the mobile device and another for handling tasks at the edge nodes. The first algorithm is executed by the mobile device, which must decide whether to process an incoming task locally or offload it to an edge node. To make this decision, the device initializes a state vector s_m that captures its historical load (i.e., the number of active queues), the current state details at time t (represented by Greek letters), and the cumulative number of bits that have been transmitted, processed, or dropped for each edge node in each time slot [2]. When a task arrives at time $t \in T$, the device takes an action $a_m(t) = \big(x_m(t),\, y_m(t)\big)$, where $x_m(t)$ specifies which queue to use (computation or transmission) and $y_m(t)$ indicates the target edge node [5]. After this decision, the device updates its state to reflect the new task delay and calculates a cost based on whether the task was successfully processed or dropped. Finally, the mobile device sends its experience—including the observed states, chosen actions, and incurred costs—to the corresponding edge node (n_m).

```
def reward_fun(delay, max_delay, unfinish_indi):
    # still use reward, but use the negative value
    penalty = - max_delay * 2

    if unfinish_indi:
        reward = penalty
    else:
        reward = - delay

    return reward
```

Listing 1: Reward function proposed by Tang et al. [2]

The second algorithm handles cases where the mobile device has offloaded a task. In this scenario, the edge node takes charge of processing the task. It begins by initializing a replay memory D_m for each device and setting up two neural networks, *Eval_Net*$_m$ and *Target_Net*$_m$. The edge node waits for incoming messages from mobile devices (the set M_n). When it receives a *parameter_request* from a device m, the edge node responds with the current parameter vector θ_m^{Eval}. Alternatively, if the edge node receives an experience tuple (containing states, actions, and costs) from a device $m \in M$, this experience is stored in the replay memory D_m. The edge node then randomly samples experiences from D_m and calculates the target Q-value $Q_{m,i}^{\text{Target}}$, using the double-DQN method. A gradient descent step is subsequently performed on *Eval_Net*$_m$ to minimize the loss function [7]. Finally, when the number of updates reaches the *Replace_Threshold*, the parameters of *Target_Net*$_m$ are refreshed by copying those from *Eval_Net*$_m$, ensuring that the target network remains current. This cooperative approach between mobile devices and edge nodes enables the system to continuously learn from past experiences and adapt dynamically to changing network conditions, thereby improving overall task offloading efficiency in MEC environments.

B. User Association and Cache Replacement

As network content caching is becoming a considerable field of research, several research are on going on the topic of user data access. In [8], a unified optimization formulation has been proposed. Optimization has been done using a greedy approach that has achieved an average delay within a $(1-1/e)$ factor. Improvement of this technique has been shown in [1] by following the two-dimensional iterative algorithm and optimal power control algorithm. A better approach has been found in [9] that uses DQN technique that highly improve the performance by minimizing delay in content caching. We followed a similar approach (with [9]) in our experiment to gain better result.

The algorithm employs an online Deep Q-Learning strategy to optimize user–AP association and cache replacement in a MEC system. At each time step t, the network state $x_b(t) \in X$ is observed, capturing critical metrics such as cache status, user associations, and channel conditions. Based on this state, an action $y_b(t) \in Y_b$ is selected using an ϵ-greedy policy: with probability $1-\epsilon$, the action that minimizes the Q-value $Q(x_b(t), y; \theta^k)$ is chosen, while with probability ϵ a random action is selected to encourage exploration. After executing the action, the algorithm observes the immediate cost $C(x_b(t), y_b(t))$ and the new network state $x_b(t+1)$, then stores the transition tuple $(x_b(t), y_b(t), C(x_b(t), y_b(t)), x_b(t+1))$ in the replay buffer O_t. A random mini-batch of these experiences is sampled to update the network parameters θ via gradient descent, and periodically, the target network parameters $\overline{\theta}$ are updated by copying the current parameters to stabilize learning. This iterative process enables the DQN to learn an optimal policy that minimizes the cost associated with user–AP association and cache replacement decisions in the MEC system.

The main contribution of this work is, in related works mentioned earlier, only considered each user associating one AP for all files. However, in general, each user usually associates with multiple APs in order to fetch different files existing in the caches of different APs.

Our main contribution is to allow each user to associate with multiple APs to fetch files. This generalization covers scenarios where different files might be cached on different APs. To achieve this, each AP has to learn the joint association and file caching replacement to minimize the delay and improve the users' Quality of Experience (QoE) including the front-haul delays of different APs. Overall, our proposed DQN algorithm offers a comprehensive solution by considering user–AP association, cache replacement, and task offloading in MEC systems.

We explicitly define a broader set of hyperparameters, such as an initial and final exploration rate with a decay schedule, thereby refining the exploration–exploitation balance. Actions now incorporate both user–AP association and cache replacement decisions, enabling holistic control over offloading tasks and file placements. The cost function is more elaborate, capturing both transmission- and caching-

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.

related penalties to accurately reflect real MEC constraints. Additionally, the Enhanced DQN includes a clearly defined process for periodically updating the target network, stabilizing learning, and emphasizes replay buffer management (with capacity limits and mini-batch sampling) to improve training efficiency. Overall, this approach not only provides a unified way to handle user requests, caching, and association, but also effectively addresses dynamic resource allocation and task offloading needs in dense MEC environments, ultimately yielding a more robust set of Q-network parameters for optimized system performance.

Algorithm 1 Enhanced DQN-Based Task Offloading with Joint Caching and User–AP Association

Initialize / Input:

- Discount factor $\gamma \in [0,1]$, initial exploration rate $\epsilon \in [0,1]$, final exploration rate $\epsilon_{\min} \in [0,1]$, decay parameter $\epsilon_{\text{decay}} \in (0,1)$
- Minibatch size M, maximum replay buffer size $|\mathcal{O}|_{\max}$, target network update frequency f_{update}, total time slots T.
- State space: $\mathcal{X} = \cdots$; Action space: $\mathcal{Y} = \cdots$
- Cost function $c_{3,t,b}(\cdots)$ and reward $r_b(t) = -c_{3,t,b}(\cdots)$.
- Primary Q-network parameters θ (randomly or pre-trained).
- Target Q-network parameters $\theta^- \leftarrow \theta$.
- Replay buffer $\mathcal{O} \leftarrow \varnothing$.

for $t = 1$ to T **do**

Observe current state $x_b(t)$.

Select action $y_b(t)$ via ϵ-greedy based on $Q(\cdot;\theta)$.

Compute cost c and reward $r_b(t) = -c$.

Observe next state $x_b(t+1)$.

Store $\big(x_b(t),\, y_b(t),\, r_b(t),\, x_b(t+1)\big)$ in $\mathcal{O}$.

Experience Replay:

(a) Sample mini-batch from $\mathcal{O}$.

(b) Compute target y_{target} for each transition.

(c) Update θ via gradient descent on $\big[y_{\text{target}} - Q(\cdot;\theta)\big]^2$.

Target Network Update (every f_{update} steps): $\theta^- \leftarrow \theta$ (or soft update).

Decay ϵ: $\epsilon \leftarrow \max\{\epsilon_{\min}, \epsilon \cdot \epsilon_{\text{decay}}\}$.

end for

Output: Q-network parameters θ for optimal offloading decisions.

IV. Proposed Method

In Figure 1 we illustrate the overview of our proposed DRL model. Consider a system with B edge nodes, such as APs, and K mobile users. The set of APs and users are denoted by $B = \{1,\ldots,B\}$ and $U = \{1,\ldots,K\}$, respectively. Each user can be served by one or more APs, depending on its connectivity to the Edge. The connectivity between users and APs is represented by a $K \times B$ binary matrix $P(t)$, where each element $P_{bk}(t) = 1$ if user k is within the coverage of AP b, and 0 otherwise.

The library of F files, denoted by $F = \{1,\ldots,F\}$, is stored in APs or the cloud, and a subset of J files, denoted by J_{files},

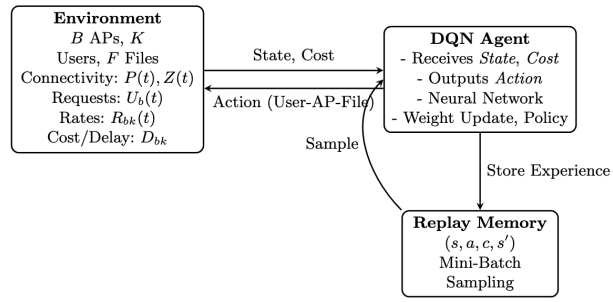


Fig. 1: Overview of the DRL model for user–AP–file associations–task offloading.

can be intelligently selected and cached in APs, where $J \ll F$. The connectivity between cached files and APs is represented by an $F \times B$ binary matrix $Z(t)$, where each element $Z_{fb}(t) = 1$ if file f is cached in AP b, and 0 otherwise.

The received requested files from users in a distributed network are denoted by $U_b(t)$, a known parameter. We defined the set of requested files for users as:

$$M = \{u_{bk} \mid u_{bk} \in \{0,1\}^F, \mathbf{u}_{bk}^T \mathbf{1} = R\}$$

where R is the number of files that each user can request, and $u_{bfk} = 1$ indicates that AP b received the request for file f from user k, while $u_{bfk} = 0$ otherwise.

The performance of file delivery to users depends not only on the cache placement strategy, but also on the specific transmission strategies used to deliver the files to the users. In the following, we will explain how to determine the delivery rates with the downlink wireless channel fading states when the requested file is cached in one or multiple associated APs.

The delivery rate at time t for user k, who is served by a single AP, b, can be expressed as [9]:

$$R_{bk}(t) = W_{bk} \log_2 \left(1 + \frac{P_{bk}|h_{bk}(t)|^2}{N_0 + I_k}\right)$$

Here, W_{bk} refers to the bandwidth assigned to user k, which is calculated by dividing the total bandwidth for base station b by the number of requests it serves. The channel coefficient between user k and AP b at time t is represented by $h_{bk}(t)$. Additionally, P_{bk} denotes the average transmission power at AP b, N_0 represents the power spectral density of noise, and I_k indicates the interference towards user k from other APs.

The cost captures the delay for users associated with AP b to download files from the AP, b, or the Cloud, which can be defined as the equation below:

$$D_{bk} = f(R_{bk}, C)$$

where C represents the computational and network constraints that influence the delay. The State, Action, and Cost Definitions in our proposed DRL is as follows:

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.

State:

$$x_b(t) = (\alpha_b(t), \sigma_b(t), G_b(t)) \in \mathcal{X}$$
$$= \{0,1\}^F \times \{0,1\}^K \times \{g_k\}_{k \in K}$$

Action:

$$y_t = (\sigma_b(t), \alpha_b(t)) \in \mathcal{Y}_b = \{0,1\}^K \times \{0,1\}^F$$

Cost:

$$c_{3,t,b}(\alpha_b(t), \sigma_b(t), G_b(t)) := \sum_{f \in F} \sum_{k \in K} \sigma_{bk}(t) \alpha_{bf}(t) u_{bfk} \frac{|s_f|}{R_{bk}(t)}$$
$$+ \sum_{f \in F} \sum_{k \in K} \sigma_{bk}(t)(1 - \alpha_{bf}(t)) u_{bfk} \left(\frac{|s_f|}{R_{bk}(t)} + D_{cf} \right).$$

Transmission Rate:

$$R_{bk} = W_{bk} \log_2 \left(1 + |h_{bk}(t)|^2 \frac{P_{bk}}{I_k + N_0} \right)$$

A. Simulation Setup

Let $\sigma_{bk}(t) \in \{0,1\}^K$ denote that AP b receives requests from user k, with a size of K. If $\sigma_{bk}(t) = 1$, user k associates with AP b; otherwise, $\sigma_{bk}(t) = 0$. The cache state at epoch t in AP b is denoted as $\alpha_{bf}(t)$. If $\alpha_{bf}(t) = 1$, file f is stored in the cache of AP b; otherwise, $\alpha_{bf}(t) = 0$.

For the implementation of independent APs, we considered path loss, bandwidth, power consumption, and interference associated with the MEC environment, making our simulation more realistic than the state-of-the art work which only considered random file access probability and file request arrival. In our current implementation, we used a randomly generated dataset consisting of the channel state information (CSI) of 4 APs, 10 users, and 10 files, generated in MATLAB. The rows represent 4 APs and the 10 columns represent the 10 CSI values corresponding to 10 users, with a transmission power of 30 dBm and a system bandwidth of 10 MHz.

Algorithm 1 (Enhanced DQN) represents the overall approach of using Deep Q-Learning (DQN) to decide how best to assign users to access points (APs) and manage the local cache in a mobile edge computing (MEC) system. The goal is to reduce the overall delay or cost associated with retrieving files. First, the algorithm represents everything happening in the system at each time (which APs are available, what's in the cache, network conditions, etc.) as a "state." At each time step, it picks a specific combination of user–AP assignments and cache replacement actions, observes how well (or poorly) these decisions work out, and saves this experience in a "replay buffer." The core DQN then uses these saved experiences to adjust its internal parameters so that, over time, it learns which actions tend to yield lower delay (or higher reward). The algorithm regularly updates a target network to help stabilize its learning, and gradually shifts from random exploration to more confident exploitation of its learned policy. This process continues until it converges on an approach that effectively decides where to offload tasks and which files to store locally in the cache.

To improve on the state-of-the-art work, our solution proposes using a deep Q-network (DQN) model in conjunction with a Markov decision process to minimize delay in content caching and improve performance. As stated in the problems statement this solution considers multiple access points to fetch files. The new DQN model follows the following approaches which differ from the original solution: In the initial step, files are randomly cached in APs using Zipf distribution and then Bernoulli distribution is used to initiate AP-user association is updated. After the initial distribution, users broadcast requests in the content delivery phase. Each AP receives the users' requests, calculates the common files from each user to AP, and gets the association relationship between users and AP. At the same time, AP knows the user's information and requested files information. Based on this information, AP refines the learning model (state, action, and rewards), broadcasts the new association results to users, and refreshes cached files accordingly.

```python
class MECEnvironment:
    def __init__(self):
        # Initialize parameters: CSI, power, interference, cache, user associations
        self.initialize_resources()

    def step(self, action):
        # Update caching and user association based on action.
        self.update_cache(action)
        self.update_association(action)
        # Compute delivery rate per user using network parameters.
        delivery_rate = self.compute_delivery_rate()
        # Calculate total delay for processing file requests.
        for user, file, unfinished in self.task_list:
            if not unfinished and self.is_cached(user, file):
                # Cached file: delay depends only on delivery rate.
                delay = file.size / delivery_rate[user]
            else:
                # Uncached or unfinished task: add penalty delay.
                delay = (file.size / delivery_rate[user]) + file.cache_delay_penalty
            total_delay += delay
        # Reward is the negative total delay (goal: minimize delay).
        reward = -total_delay
        # Get updated state and check if the process is done.
        observation = self.get_current_state()
        done = self.check_termination()
        return observation, reward, done
```

Listing 2: Concept of MEC Environment with Joint Task Offloading, Caching, and User Association

In our framework, the state encapsulates key wireless resources and network conditions, including channel state information (CSI), transmit power, interference levels, user–AP associations, and cache status. The CSI is converted from complex channel gains into power values and used alongside transmit power and noise to calculate the effective delivery rate

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.

through a capacity-like logarithmic formula. This delivery rate then determines the delay incurred in serving user requests, where cached files incur a lower delay and non-cached files include an additional penalty. The reward for each action is directly based on this total delay, enabling the reinforcement learning agent to minimize delay by jointly optimizing user association and cache replacement decisions in the MEC environment. The cost of the DQN model represents the objective of the framework, which is simultaneously minimizing the network delay and other costs, such as energy while achieving a good Quality of user Experience.

V. Evaluation

In this section, we present our results, outlining the improvements made in our research. Additionally, we compare our approach with the baseline (state-of-the-art) and provide a thorough examination of the corresponding outcomes. We investigated scenarios involving ten mobile devices and four edge nodes. The proposed algorithm's convergence is conducted across various neural network hyperparameters and algorithmic configurations.

In the state-of-the-art work [2], the researchers used several metrics to compare the results of their task offloading solution to other previously proposed solutions. Figures 2, and 4 all demonstrate how the state-of-the-art solution converges. All these graphs depict the cost against the number of episodes. Each figure varies by metrics. Figure 2 utilizes different learning rates (0.01, 0.001, and 0.0001) as its changing metric between graphs. From these, the smaller learning rate the faster the convergence but potentially at the cost of higher variance. Figure 4 change metric is different batch sizes (2, 8, and 32) As the figure depicts the lower the batch size the faster the convergence. Figure 6 depicts the dropped task ratio vs task arrival probability (TAP), and the number of mobile devices. As for the other two charts, figure 6 shows an increase in the number of the respective metric shows an increase. TAP increase slowly increases till around 0.3 then afterward increases at a steady rate. The number of Mobile devices has the same effect on the ratio of dropped tasks where it slowly increases until around 60 mobile devices and then converges upward. In the last figure we compare the average delay against the same metrics in figure 6. All three of the charts in figure 5 depict an increase in delay as the metrics increase. We can see the graphs converge upwards at around 0.2, 1.75, and 40 respectively for TAP, task deadline, and the number of mobile devices.

Our proposed framework focuses on a different setting: ultra-dense MEC networks, where there are many access points and conventional methods are ineffective due to long transmission delays. The proposed algorithm in our study is designed to meet real-time offloading needs and plays a crucial role in fifth-generation (5G) networks. As seen in both the *Existing Problems Solution* and *Proposed Solution* sections, the reward functions are slightly different. The reward and cost functions in the state-of-the-art solution depend solely on delay. Thus, we compare the cost of the state-of-the-art solution to the delay in our proposed solution.

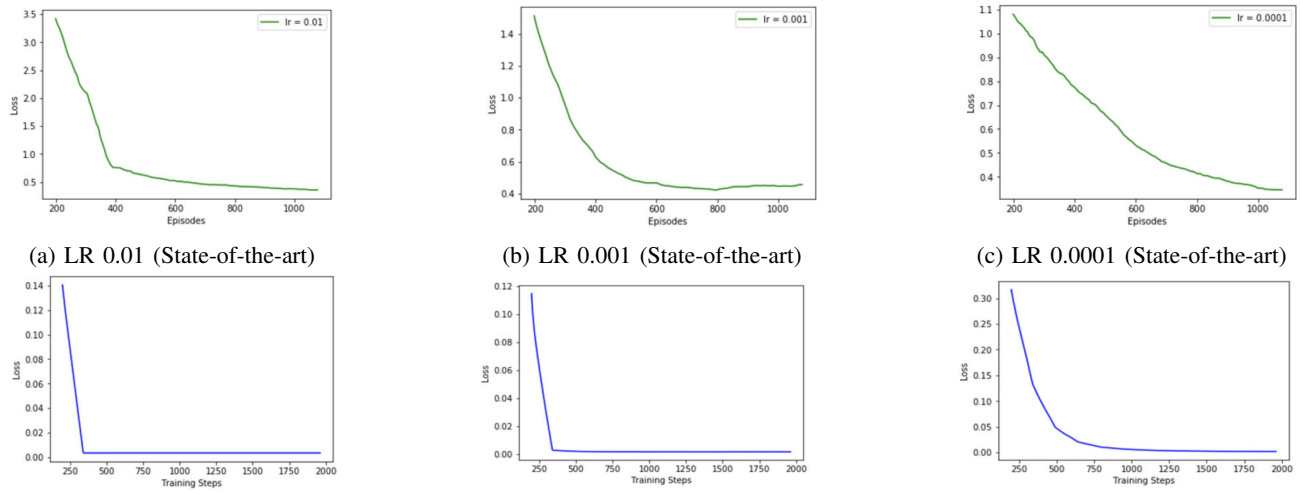
In Figure 2 a comparison between our proposed framework and the state-of-the-art work has been presented that shows convergence with loss using the same learning rates (0.01, 0.001, and 0.0001). Both metrics illustrate convergence for different learning rates, depicting the loss against the number of episodes or training steps. Regardless of the learning rate used, our proposed solution shows a significant improvement over the state-of-the-art solution. The next set of comparisons follows the same pattern as the previous one, but instead of learning rates, we use batch sizes of 2, 8, and 32 in Figure 4. Despite this modification, our solution continues to outperform the cutting-edge methods proposed by the authors. We also compared the cost (for state-of-the art framework) which is referred to as the delay in our approach for the same learning rates (0.01, 0.001, and 0.0001) shown in Figure 3.

For the state-of-the-art approach in Figure 3, we see how changes in learning rate can affect the speed and stability of convergence. A higher learning rate, such as 0.01, may cause more oscillations or slower stabilization, whereas a smaller learning rate (e.g., 0.0001) can lead to more stable but potentially slower progress. In contrast, the proposed framework shows its convergence behavior under the same three learning rates. Overall, we can observe that the proposed framework tends to converge more quickly and smoothly, e.g., at a learning rate of 0.001, the proposed approach appears to reach a stable lower cost (or higher reward) faster than the state-of-the-art [2] method.

The next metric to compare is the Average Delay vs Task Arrival Probability (TAP) and the number of mobile devices. Figure 5c and 5d shows that the results from our proposed solution do not start to significantly increase until around 0.7 for TAP and 95 mobile devices. In contrast, the state-of-the-art solution in Figure 5a and 5b starts to increase around 0.2 for TAP and 20 mobile devices. This demonstrates that our solution provides an improvement over the state-of-the-art approach for this metric. In Figure 6a, we observe the last metric, which compares the Task Dropped Ratio vs Task Arrival Probability. If we examine Figure 6b, we see that in state-of-the-art work, the task dropped ratio starts increasing from 0.3 in a logarithmic manner and eventually converges at 0.7, indicating a high task drop rate. On the other hand, in our solution, the increase starts at a similar point, but the convergence remains around 0.3, signifying that more query requests are answered in comparison to the state-of-the-art work.

Overall, these results demonstrate significant performance boost by replacing the traditional reinforcement model with a DQN while incorporating a Markov learning model and considering multiple access points for users. This demonstrates that our model achieves the high quality of user experience that we set out to accomplish.

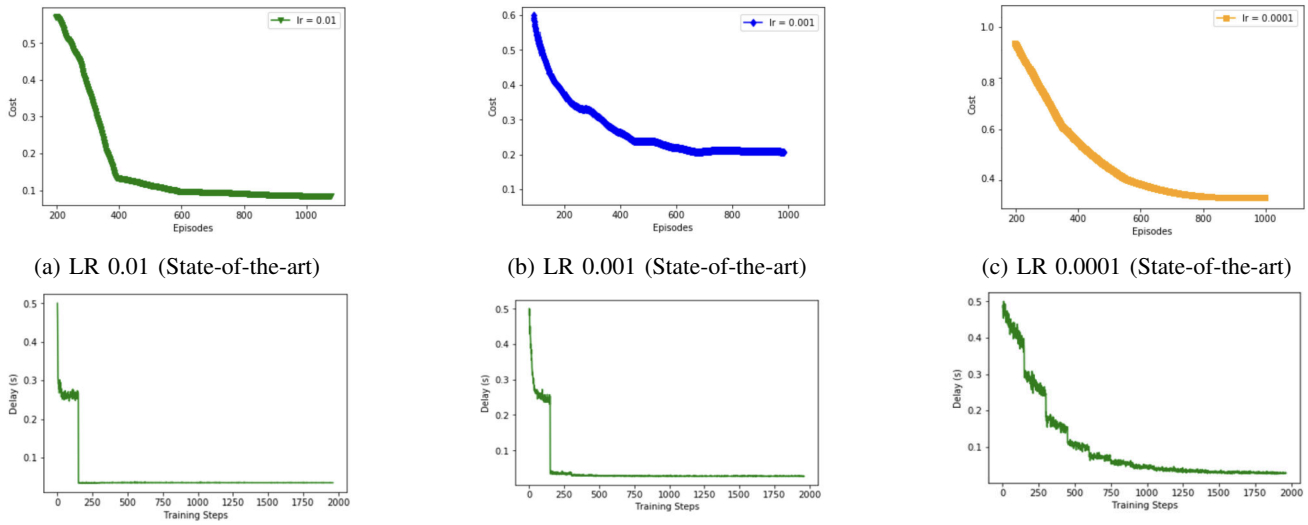
Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.



(a) LR 0.01 (State-of-the-art) (b) LR 0.001 (State-of-the-art) (c) LR 0.0001 (State-of-the-art)

(d) LR 0.01 (Proposed framework) (e) LR 0.001 (Proposed framework) (f) LR 0.0001 (Proposed framework)

Fig. 2: Comparison of convergence of algorithms under different learning rates.



(a) LR 0.01 (State-of-the-art) (b) LR 0.001 (State-of-the-art) (c) LR 0.0001 (State-of-the-art)

(d) LR 0.01 (Proposed framework) (e) LR 0.001 (Proposed framework) (f) LR 0.0001 (Proposed framework)

Fig. 3: Comparison of convergence of cost vs delay used for the reward function of the DQN model under different learning rates.

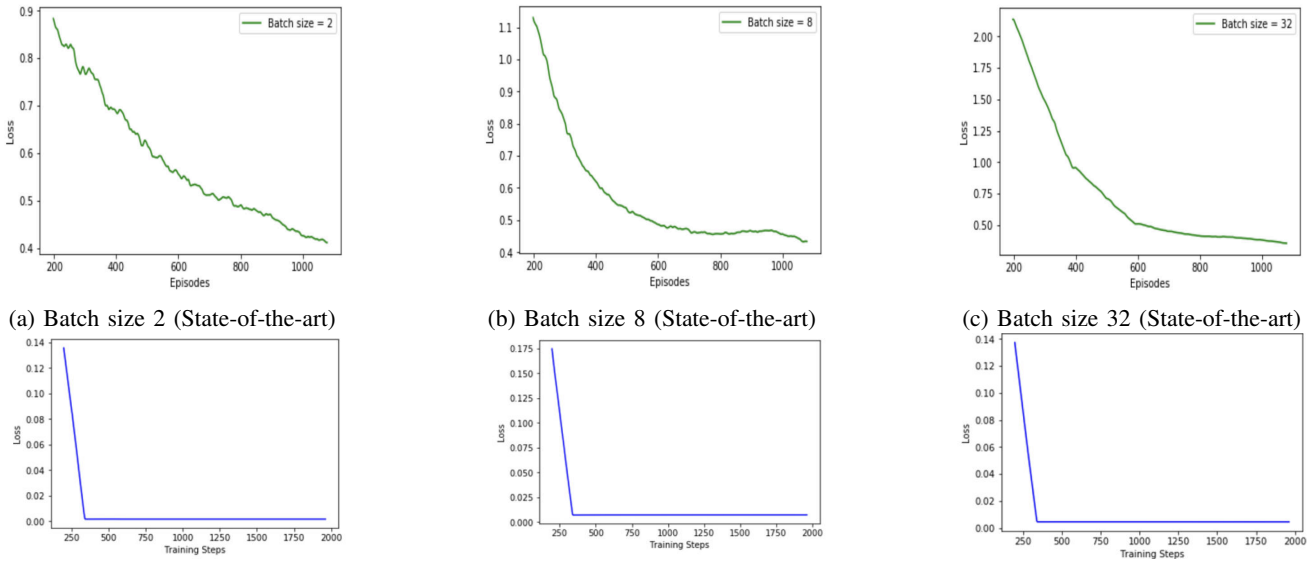
VI. Conclusion

In this work, we introduce a deep reinforcement learning framework that jointly addresses task offloading, caching, and user–AP association in Mobile Edge Computing (MEC) systems. Unlike previous approaches that become infeasible when numerous base stations exhibit diverse transmission qualities, our method employs a dual-algorithm strategy. On the mobile device side, a state vector capturing historical load, transmission, processing, and dropping metrics is used to decide whether an incoming task should be processed locally or offloaded to an edge node via an action defined by the choice of queue and target AP. Concurrently, at the edge node, a Double DQN-based solution is implemented that leverages replay memory and two neural networks—*Eval Net* and *Target Net*—to dynamically optimize file caching and transmission strategies based on downlink channel conditions and interference. This integrated approach allows each user to be served by multiple APs and intelligently caches a subset of files to further minimize delay. Experimental results demonstrate that our framework significantly improves performance across key metrics compared to baseline methods. We believe this research offers a robust and adaptable solution for efficient task offloading in next-generation MEC systems.

References

[1] F. Guo, H. Zhang, X. Li, H. Ji, and V. C. M. Leung, "Joint optimization of caching and association in energy-harvesting-powered small-cell networks," *IEEE Transactions on Vehicular Technology*, vol. 67, no. 7, pp. 6469–6480, 2018.

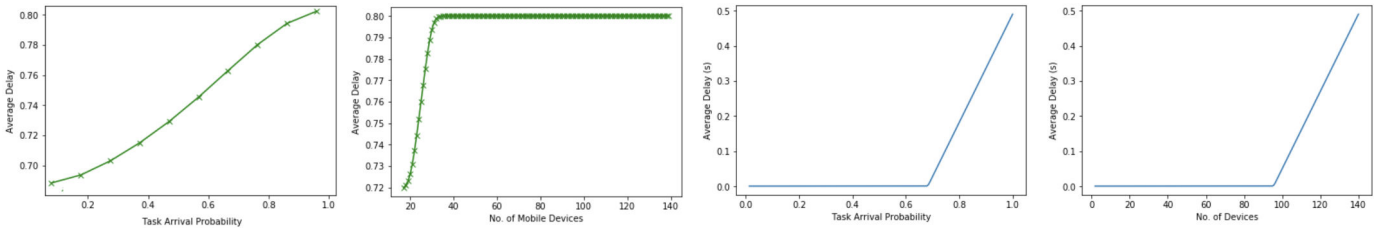
Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.



(a) Batch size 2 (State-of-the-art) (b) Batch size 8 (State-of-the-art) (c) Batch size 32 (State-of-the-art)

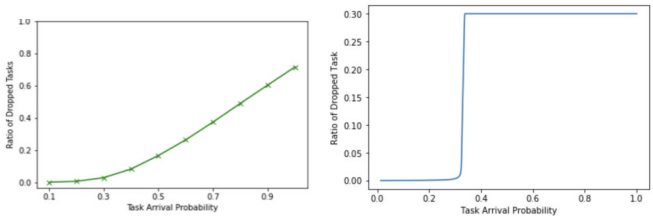
(d) Batch size 2 (Proposed framework) (e) Batch size 8 (Proposed framework) (f) Batch size 32 (Proposed framework)

Fig. 4: Comparison of convergence of algorithms under different batch sizes.



(a) Average delay vs task arrival probability (state-of-the-art) (b) Average delay vs no. of devices (state-of-the-art) (c) Average delay vs task arrival probability (proposed framework) (d) Average delay vs no. of devices (proposed framework)

Fig. 5: Comparison of delay ratio vs task arrival probability, and number of mobile devices between our approach and the state-of-the-art framework.



(a) State-of-the-art Solution (b) Proposed Solution

Fig. 6: Comparison of state-of-the-art and proposed solution in terms of Dropped tasks ratio vs task arrival probability.

[2] M. Tang and V. W. Wong, "Deep reinforcement learning for task offloading in mobile edge computing systems," *IEEE Transactions on Mobile Computing*, vol. 21, no. 6, pp. 1985–1997, 2022.

[3] J. Wang, L. Zhao, J. Liu, and N. Kato, "Smart resource allocation for mobile edge computing: A deep reinforcement learning approach," *IEEE Transactions on Emerging Topics in Computing*, vol. 9, no. 3, pp. 1529–1541, 2021.

[4] T. Alfakih, M. M. Hassan, A. Gumaei, C. Savaglio, and G. Fortino, "Task offloading and resource allocation for mobile edge computing by deep reinforcement learning based on sarsa," *IEEE Access*, vol. 8, pp. 54 074–54 084, 2020.

[5] D. Zeng, L. Gu, S. Pan, J. Cai, and S. Guo, "Resource management at the network edge: A deep reinforcement learning approach," *IEEE Network*, vol. 33, no. 3, pp. 26–33, 2019.

[6] J. Liu, Y. Mao, J. Zhang, and K. B. Letaief, "Delay-optimal computation task scheduling for mobile-edge computing systems," in *2016 IEEE International Symposium on Information Theory (ISIT)*, 2016, pp. 1451–1455.

[7] S. Jošilo and G. Dán, "Selfish decentralized computation offloading for mobile cloud computing in dense wireless networks," *IEEE Transactions on Mobile Computing*, vol. 18, no. 1, pp. 207–220, 2019.

[8] M. Dehghan, A. Seetharam, B. Jiang, T. He, T. Salonidis, J. Kurose, D. Towsley, and R. Sitaraman, "On the complexity of optimal routing and content caching in heterogeneous networks," in *2015 IEEE Conference on Computer Communications (INFOCOM)*, 2015, pp. 936–944.

[9] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, and M. Bennis, "Performance optimization in mobile-edge computing via deep reinforcement learning," 2018. [Online]. Available: https://arxiv.org/abs/1804.00514

Authorized licensed use limited to: University of Dayton Libraries. Downloaded on March 18,2026 at 14:29:28 UTC from IEEE Xplore. Restrictions apply.